

EN.605.603: AI/ML Ops

Term: Summer 2026

Term Project: Final Capstone Report

Project Name: When and Where You Compute Is What You Emit: A Carbon-Aware MLOps Pipeline for AI Infrastructure

Team Members: Andrew Saifnoorian

Abstract

Data centers are on pace to become one of the fastest-growing sources of global electricity demand, with AI compute alone projected to push consumption past 800 TWh by 2028, on the order of a mid-sized country's annual draw (IEA, 2025). This paper presents a carbon-aware MLOps pipeline that treats *when* and *where* a workload runs as a decision variable rather than a fixed constraint, and operationalizes that decision as a single actionable signal: a Green Window Probability and a RUN WORKLOAD / WAIT / ADJUST recommendation. Three machine learning components were designed, trained, and deployed behind a unified inference layer, the Carbon-Opportunity Signal: an XGBoost classifier that labels a forecast window as low- or high-carbon (86.1% accuracy, $F1 = 0.87$, meeting the project's target), an XGBoost regressor that estimates national data-center power-capacity growth from AI-adoption indicators, and a K-Means ($k = 6$) clustering step that assigns countries to carbon-sustainability archetypes. The pipeline survived a real-world MLOps failure mode mid-build: two of six proposed datasets were deprecated after the proposal was approved, and the substitution is documented rather than hidden. The regression component fell short of its target (country-grouped cross-validated $R^2 = 0.16$ against a target of 0.70), and that result is reported honestly alongside the leaky, over-optimistic number a naive random split would have produced ($R^2 = 0.51$), because the gap between the two is itself the paper's clearest lesson in MLOps evaluation discipline. The complete signal is served through a local FastAPI development server with an interactive OpenAPI (Swagger) surface and an Azure ML Managed Online Endpoint, closing the loop from raw grid data to a deployable, explainable scheduling recommendation. All modeling code, data-ingestion scripts, trained artifacts, and deployment configuration are version-controlled in the project repository, github.com/andrewsaifnoorian/carbon-aware-mlops.

Table of Contents

Table of Contents	2
1. Introduction and Motivation	3
2. Data Landscape	4
3. System Architecture	4
4. Data Engineering: A Real-World Deprecation Challenge	6
5. Algorithm Selection and Methodology	6
6. Experimental Results	8
7. The Carbon-Opportunity Signal: From Predictions to a Recommendation	12
8. Serving the Signal: FastAPI and the OpenAPI (Swagger) Surface	13
9. Live API Demonstration	14
10. Deployment: Azure ML Managed Online Endpoint	16
11. MLOps Lifecycle: Monitoring, Drift Detection, and Retraining	16
12. Discussion: Limitations and Why Honest Reporting Matters	17
13. Evaluation, Feedback, and Outcomes	17
14. Conclusion and Future Work	19
References	19
Appendix A: Reproducing the Pipeline	20
Appendix B: Carbon-Opportunity Signal API Reference	20
Appendix C: Repository Structure	21

1. Introduction and Motivation

1.1 The Hidden Cost of AI Compute

Every inference request, training run, and batch job carries an invisible carbon cost that is a function of two variables the requester almost never controls or even sees: *when* the workload runs, and *where* on the electrical grid it runs. A GPU cluster drawing power from a region running 80% coal at 6 PM on a July weekday emits several times more CO₂ per kilowatt-hour than the same cluster running at 2 AM in a region with high wind penetration. As AI workloads scale, this gap compounds: the International Energy Agency projects that global AI data-center electricity consumption will surpass 800 TWh by 2028, a demand level comparable to the annual consumption of France (IEA, 2025). Most of that growth is being absorbed by existing or newly-built infrastructure with little regard for the carbon intensity of the grid it draws from at the moment it draws it.

This project treats that gap as an MLOps problem rather than a policy problem. Rather than asking organizations to build new low-carbon data centers, a capital-intensive, multi-year undertaking, it asks a narrower, immediately actionable question: given the compute infrastructure that already exists, can a model learn to identify low-carbon windows and locations well enough to shift discretionary workloads toward them, and can that model be operationalized as a production service rather than a one-off notebook analysis?

1.2 Research Question and Approach

Research question: How might we accurately identify low-carbon compute windows and locations across the electrical grid, and operationalize those predictions to support carbon-aware workload scheduling? The approach taken here decomposes the question into three supervised and unsupervised learning problems that are trained independently but combined at inference time into one recommendation:

- **Classification:** is a given region, at a given hour, day-of-week, and month, a low-carbon (“green”) or high-carbon (“red”) compute window?
- **Regression:** what national data-center power-capacity growth does a country's AI-adoption trajectory imply, to prioritize which regions carbon-aware scheduling matters most for?
- **Clustering:** which carbon-sustainability archetype does a country or facility belong to, to add locational context beyond the time-of-day signal alone?

The three outputs are fused into a single artifact, the Carbon-Opportunity Signal: a lightweight inference layer that returns a Green Window Probability and a plain-language recommendation, RUN WORKLOAD, WAIT, or ADJUST, that a human operator or an automated orchestrator can act on directly. Section 3 details the system architecture; Sections 5 and 6 walk through algorithm selection and results for each of the three models; Section 7 describes how the outputs are fused; Section 8 covers the serving surface and the OpenAPI/Swagger contract; Section 9 presents a live API demonstration; and Section 10 covers deployment to Azure ML.

2. Data Landscape

The pipeline draws on four data sources spanning three levels of granularity: an hourly, US-grid-region-level electricity monitor; a country-year panel of AI-adoption and infrastructure indicators; a facility-level inventory of US data centers; and a country-year panel of long-run sustainable-energy indicators.

- **EIA-930 Hourly Electric Grid Monitor** (U.S. Energy Information Administration): hourly net generation by fuel source and hourly CO₂ emissions intensity, generated and consumed, across 12 US grid balancing regions. The processed table (`eia930_hourly.csv`) holds roughly 379,000 rows spanning 2023 through 2026. This is the classifier's primary training table.
- **AI Readiness and Growth, 150 Countries** (GAID compilation, Kaggle): country-level AI Index sub-scores (Infrastructure, Talent, Commercial, Development, Government Strategy, Operating Environment, Intensity, Research, Scale) alongside National AI Cluster Power Capacity (MW), the regression target. The processed regression table holds 126 country-year rows across 34 countries.
- **ICARUS-DC** (Johns Hopkins APL / NSF): 354 U.S. data-center facilities with operator, state, and power capacity, each mapped to an EIA-930 grid region (`facility_lookup.csv`) to support facility-level recommendations.
- **Global Data on Sustainable Energy** (Kaggle): country-year renewable share, low-carbon electricity percentage, and CO₂ emissions, used as clustering features alongside AI infrastructure indicators (29 countries after dropping rows with missing features).

Raw downloads are intentionally excluded from version control (`data/raw/` is gitignored) and are regenerated on demand by the four `ingest_*.py` scripts; only the processed, model-ready tables in `data/processed/` and the trained artifacts in `models/` (plus their metrics JSON) are treated as build outputs worth persisting. This mirrors standard MLOps practice of separating raw data acquisition, which is slow, external, and subject to source drift, from the fast, deterministic feature-engineering and training steps that consume it.

3. System Architecture

The pipeline is organized as four stages, each a discrete, independently re-runnable script: ingestion, training, signal computation, and serving. Figure 1 shows the full data flow from raw downloads through the two serving paths, a local FastAPI development server and an Azure ML Managed Online Endpoint, both of which execute the identical `carbon_signal.py` inference code; only the hosting layer changes between development and production. A monitoring-and-retraining feedback loop closes the diagram: because the EIA-930 feed is continuously refreshed, the classifier is the component most exposed to data drift, and the pipeline is designed so that a scheduled re-ingest, retrain, and blue/green redeploy can be run without touching the inference or serving code.

Figure 1 End-to-end carbon-aware MLOps pipeline

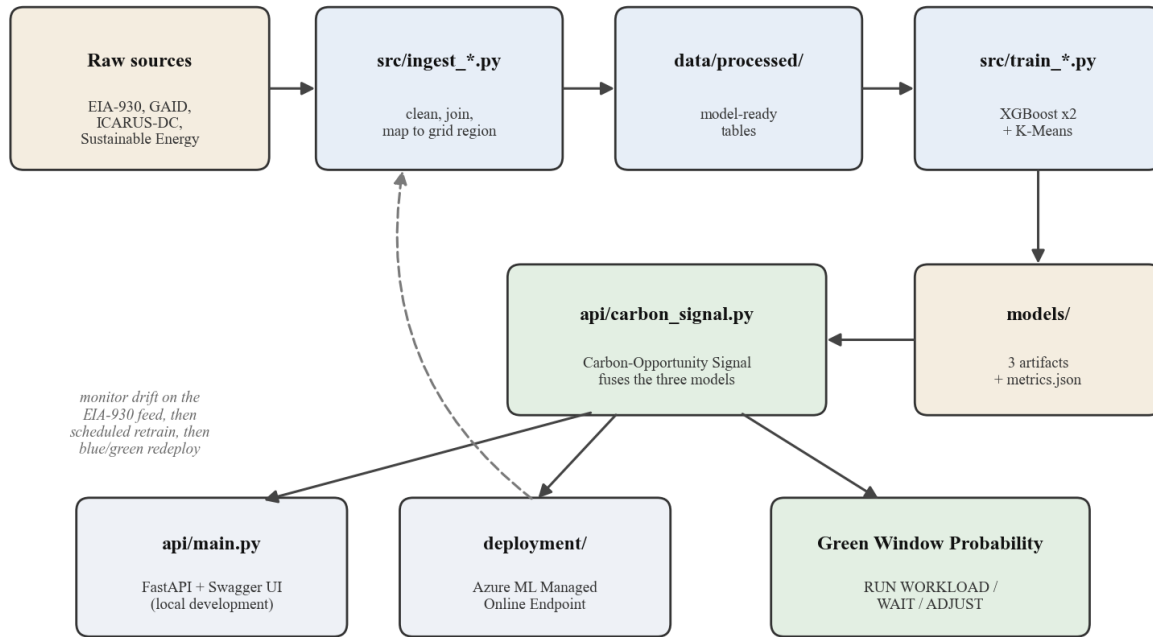


Figure 1. End-to-end pipeline: raw sources are ingested into model-ready tables, three models are trained independently, and the Carbon-Opportunity Signal layer combines their outputs into a single recommendation served identically from FastAPI or Azure ML. The dashed arc is the monitoring and retraining loop.

Table 1 maps each pipeline stage to the MLOps lifecycle phase it implements and the concrete tooling used, following the four-part framing introduced in the project's algorithm-selection deliverable (business analytics, model development, model vetting, model operations).

Lifecycle phase	Pipeline stage	Tooling / artifacts
Data ingestion	src/ingest_*.py	Kaggle API, EIA-930 bulk files, ICARUS-DC clone; pandas joins on grid region
Feature engineering	src/ingest_*.py outputs	cyclical (sin/cos) time encodings, region one-hots, demand z-score, log1p target
Training / experimentation	src/train_*.py	XGBoost + scikit-learn; RandomizedSearchCV, TimeSeriesSplit, GroupKFold
Model vetting	models/*_metrics.json, src/make_figures.py	held-out and grouped-CV metrics, confusion matrix, elbow/silhouette, leakage comparison
Packaging / deployment	deployment/	Azure ML endpoint.yml / deployment.yml, conda.yaml, score.py, model_package/
Serving / consumption	api/main.py, Azure ML endpoint	FastAPI + Uvicorn, OpenAPI 3.1 / Swagger UI, Postman collection, demo client
Monitoring / retraining	scheduled re-run of ingest + train + deploy	drift checks on the EIA-930 feed, blue/green redeployment

4. Data Engineering: A Real-World Deprecation Challenge

Six datasets were cited and approved in the Preliminary Proposal and Algorithm Selection deliverables (both graded 100/100, July 14, 2026). By the time implementation began, two of the six had been removed from Kaggle entirely, and a third, Electricity Maps' hourly carbon-intensity CSVs, the dataset the proposal explicitly called “the operationally differentiating dataset,” had moved its hourly data behind a paid API and account requirement. Silent dataset deprecation mid-project is a legitimate and under-discussed MLOps failure mode: pipelines built against a specific external source are only as durable as that source's availability guarantees, and a production system needs a documented substitution path rather than a broken build. Table 2 documents each substitution rather than quietly absorbing it.

#	Proposal source	Status at build time	Used instead
1	Kaggle: Global Data Center Dataset	404, removed from Kaggle	AI Readiness & Growth (150 Countries): National AI Cluster Power Capacity (MW)
2	Kaggle: Global AI & Data Consumption 2025	404, removed from Kaggle	Same GAID compilation, nine AI Index sub-scores as adoption features
3	Electricity Maps hourly carbon intensity	Free tier removed; now paid API/account	EIA-930 Hourly Grid Monitor: free, public, U.S. government, no login
4	ICARUS-DC (GitHub)	Available	Used as cited, cloned directly from GitHub
5	LBNL 2024 U.S. Data Center Energy Usage Report	Site blocks automated fetches	Cited for narrative grounding only (never a raw modeling input)
6	Global Data on Sustainable Energy (Kaggle)	Available	Used as cited

The EIA-930 substitution turned out to be a strict upgrade rather than a compromise: it is authoritative (official U.S. government data), free with no registration, and geographically aligned with the ICARUS-DC facility data, since both are U.S.-grid-centric, arguably a cleaner join than the originally proposed Electricity Maps plus ICARUS-DC pairing would have produced. Likewise, the AI Readiness and Growth compilation turned out to cover both missing roles, the data-center-capacity target and the country-level AI-adoption features, in a single source, which is a cleaner join than stitching together two separately-sourced Kaggle datasets would have been. The full provenance trail, including which processed table each source feeds, is kept in `docs/DATA_SOURCES.md` in the repository.

5. Algorithm Selection and Methodology

Each of the three deliverables was assigned a distinct algorithm based on the structure of its underlying data and the operational requirement that every model run efficiently behind a low-latency Azure Managed Endpoint, support periodic retraining, and expose feature importance so the run/wait/adjust recommendation can be explained rather than treated as a black box.

5.1 Classifier: XGBoost

The Green Window classifier uses gradient-boosted trees (XGBoost) to label a forecast window as green or red from renewable percentage, a region-relative demand z-score, region one-hot encodings, and cyclical (sine/cosine) encodings of hour-of-day, day-of-week, and month. The label is defined *relative to each region's own historical median* carbon intensity, computed on the training period only: green = 1 when the hour's carbon-intensity-of-generation is below that region's median, else red = 0. A region-relative threshold is used because baseline carbon intensity varies enormously by region; the operational question is not “is this globally clean” but “is this a good time to run compute *for this facility's grid*.” Evaluation uses a chronological 80/20 split (the last 20% of the timeline is held out), which simulates the real forecasting task of predicting a future window rather than an already-seen one. Hyperparameters were tuned with RandomizedSearchCV (20 iterations, TimeSeriesSplit with four folds); the selected model uses 400 estimators, max depth 6, learning rate 0.1, subsample 0.7, colsample_bytree 0.8, and min_child_weight 5.

Logistic regression was retained only as an interpretability baseline; it cannot model the nonlinear hour-times-renewable-availability interaction central to the research question. A deep neural network was rejected: after country/region-level aggregation the assembled tables are modest in size, insufficient to train a deep model without severe overfitting, and a boosted-tree model is materially cheaper to serve at low latency from a managed endpoint.

5.2 Regressor: XGBoost

The capacity-growth regressor predicts national data-center power capacity (MW, log-transformed via $\log_{10}$) from AI-adoption features: the AI Index overall score, the scale sub-score, and survey year. The same reasoning that favored boosted trees for classification applies here: capacity growth is unlikely to scale linearly with AI-adoption scores (diminishing or accelerating returns are plausible at the high end), and the panel is small, tabular, and assembled from multiple joined sources with mixed types, conditions XGBoost handles natively without extensive preprocessing. The model is deliberately shallow and heavily regularized (max depth 1, learning rate 0.05, 100 estimators, `reg_lambda` 5.0) given the small sample. Random Forest was considered as a lower-variance alternative but produces a larger artifact, which is less favorable for endpoint size and latency.

Critically, the evaluation was designed to expose leakage rather than hide it. The panel has 126 rows but only 34 distinct countries, because the AI Index sub-scores are computed once per country and repeated across every year of capacity data for that country. A naive random 80/20 split lets the same country appear in both train and test, inflating R^2 through memorization rather than generalization. The honest estimate uses GroupKFold by country (five folds), so no country appears in both train and test. Both numbers are reported in Section 6.2.

5.3 Clustering: K-Means (k = 6)

K-Means was retained from the proposal to assign countries to carbon-sustainability archetypes using renewable share, low-carbon electricity percentage, CO₂ emissions, energy intensity, log data-center capacity, and AI infrastructure score, all standardized before fitting. K-Means was preferred over hierarchical clustering and DBSCAN because the archetype features are continuous and approximately bounded after scaling, a six-archetype target maps naturally onto a business-interpretable fixed k, and the algorithm scales cleanly and predictably as an automatically retrained pipeline step, a property hierarchical clustering and DBSCAN do not share as k grows or shrinks with new incoming data. The trade-off accepted was the assumption of roughly spherical, similarly-sized clusters and sensitivity to feature scale and outliers, both mitigated by standardizing inputs and using 25 k-means++ restarts. Centroids are labeled with human-readable archetype names from a rank-based tercile of each centroid's standardized profile (fossil-heavy / mixed-grid / renewable-rich, and so on).

6. Experimental Results

6.1 Classification: Green Window Probability

The classifier was evaluated on a 75,798-row held-out test split (of 378,986 total rows) and comfortably met both proposal targets: 86.1% accuracy against a target of at least 85%, and F1 = 0.87 against a target of at least 0.80. ROC-AUC reached 0.94. Figure 2 shows the confusion matrix; the model's error is roughly balanced between false reds (4,531 green windows misclassified as red, a missed-opportunity cost) and false greens (5,990 red windows misclassified as green, the more operationally costly error since it would schedule a workload into a high-carbon window under a false green light).

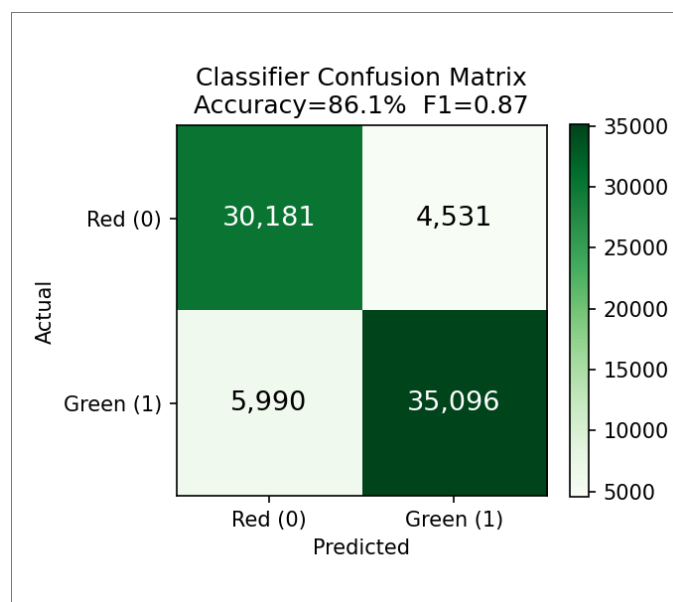


Figure 2. Classifier confusion matrix on the 75,798-row held-out test set. Accuracy 86.1%, F1 0.87, both proposal targets met.

Figure 3 shows the top feature importances. Region indicators dominate: the Northwest (`region_NW`), Central (`region_CENT`), and California (`region_CAL`) one-hot features are the three strongest

predictors, ahead of `renewable_pct` itself. This is an interpretable and, in retrospect, unsurprising result: which grid you are on is a stronger signal than the instantaneous renewable percentage alone, because each region's baseline fuel mix (hydro-heavy Northwest versus coal-heavier Central) sets a very different carbon-intensity floor and ceiling before time-of-day effects are even considered. Per-region green/red thresholds were fit individually, ranging from roughly 0.4 kg CO₂/kWh in California to 1.1 in the Midwest, rather than applying one global cutoff, precisely because of this baseline gap.

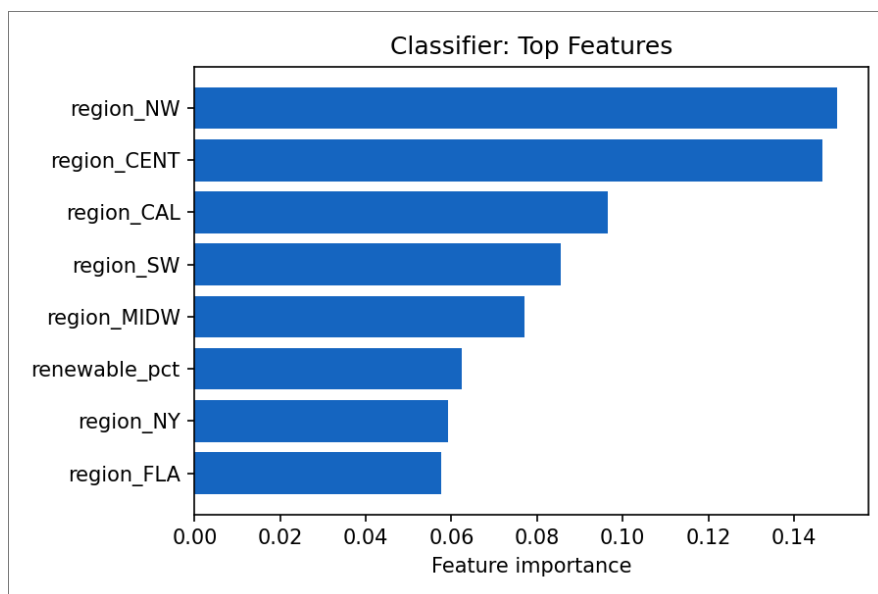


Figure 3. Top eight classifier feature importances. Which grid region a facility sits on outweighs the instantaneous renewable percentage as a predictor of window carbon intensity.

6.2 Regression: National Infrastructure Capacity, and the Leakage Lesson

The regression target, national data-center power capacity (MW) from AI-adoption features across 34 countries and 126 country-year rows, did not meet its target, and that result is reported without qualification: country-grouped cross-validated $R^2 = 0.16$ against a target of at least 0.70. What makes this result worth a dedicated figure rather than a footnote is why it fell short, and what a naive evaluation would have hidden. A random 80/20 train-test split on this panel, which places different years of the same country in both the train and test sets, produces $R^2 = 0.51$ (MAE 7.19 MW), because the model can partially memorize a country's identity from correlated features seen in another year of that same country's data rather than genuinely learning the AI-adoption-to-capacity relationship. A country-grouped 5-fold split, where every row for a given country is held entirely out of training in at least one fold, removes that leakage and drops the honest estimate to $R^2 = 0.16$, with two of the five folds posting negative R^2 (the model does worse than predicting the mean for held-out countries). Figure 4 makes the three numbers, leaky split, honest split, and target, visible side by side.

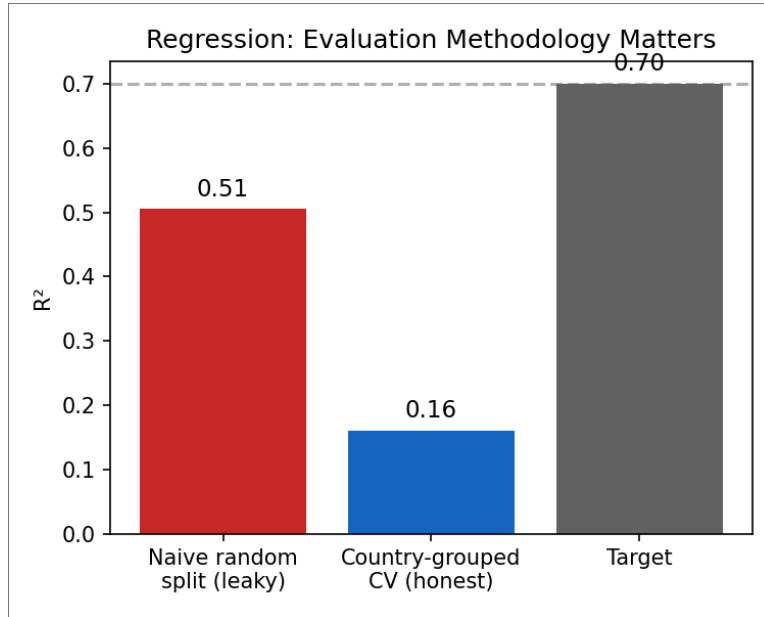


Figure 4. Evaluation methodology matters more than the model here: the naive random split (leaky) overstates generalization by roughly 3x relative to the honest, country-grouped cross-validated estimate.

With only 34 countries and 126 rows, and just three features carrying meaningful importance (overall score 0.39, scale score 0.31, year 0.30), the dataset is simply too small and too coarse-grained for a tree ensemble to learn a generalizable country-to-country relationship: national AI-adoption scores and data-center capacity are both driven by deeper structural factors (GDP, existing grid infrastructure, hyperscaler siting decisions) that are not present as features. Reporting $R^2 = 0.16$ honestly, rather than the $R^2 = 0.51$ a less careful evaluation would have surfaced, was judged more valuable to the project than hitting the target number on a compromised methodology: a stakeholder who deployed a scheduling system on the belief that the capacity regressor generalizes to countries it never saw during training would have been badly misled by the leaky number. The deployed model is refit on all available data, standard practice once the evaluation methodology has been fixed, and is exposed only as a coarse prioritization signal, never as a precise forecast.

6.3 Clustering: Carbon-Sustainability Archetypes

K-Means was fit for $k = 2$ through 9 on 29 countries; the elbow and silhouette curves in Figure 5 both flatten meaningfully after $k = 6$, and $k = 6$ was retained as specified in the proposal, yielding a silhouette score of 0.22, a modest but usable degree of cluster separation, consistent with the reality that sustainability profiles form a spectrum rather than cleanly separated groups.

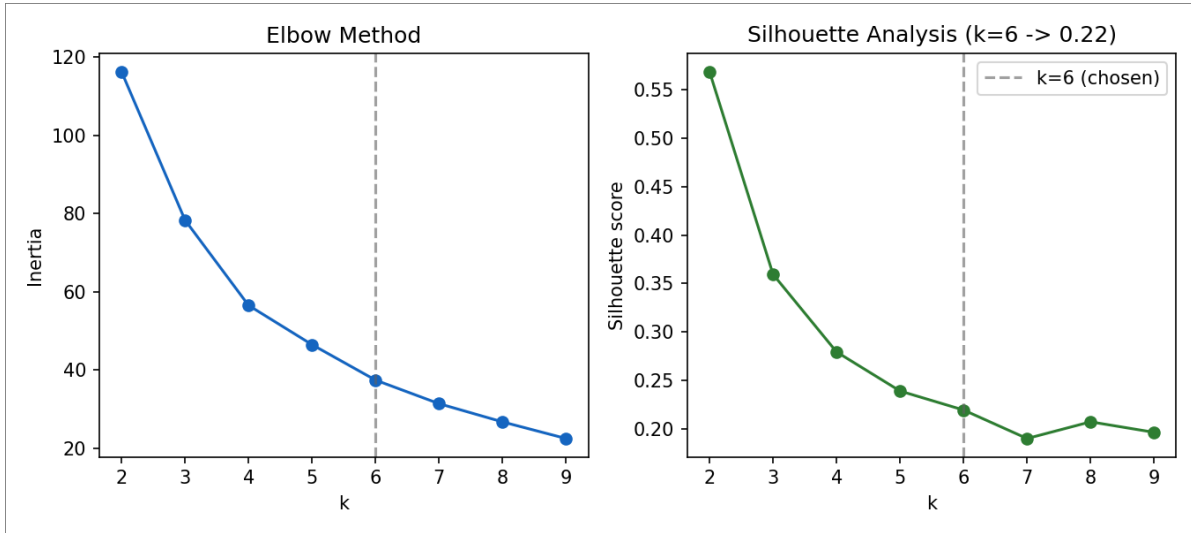


Figure 5. Elbow method (left) and silhouette analysis (right) across $k = 2$ to 9. Both curves flatten past $k = 6$, supporting the proposal's fixed six-archetype target.

Table 3 and Figure 6 summarize the six resulting archetypes, hand-labeled from their standardized centroids. They range from renewable-rich, low-carbon, small-capacity (Iceland, alone, at 81% renewable share and essentially zero net emissions) to high-emissions fossil-heavy, hyperscale-capacity (the United States, alone, with a data-center capacity centroid far above every other cluster). China forms its own singleton cluster as the sole high-emissions mixed-grid, hyperscale-capacity country. The remaining three clusters are more populated: eleven fossil-heavy, mid-capacity countries (including Australia, India, and Saudi Arabia); five renewable-rich, mid-capacity countries (Brazil, Canada, and the Nordic states); and ten mixed-grid, small-capacity European countries. The three singleton clusters are a direct, honest consequence of how structurally distinct those countries' energy-and-compute profiles are from every other country in the sample, not a clustering artifact to be explained away.

Cluster	Archetype label	Renew. %	Low-carbon %	DC capacity (log1p MW)	Members
C0	fossil-heavy, high-carbon, mid-capacity	9.7	13.3	2.20	11 (Australia, India, Japan, ...)
C1	renewable-rich, low-carbon, mid-capacity	46.1	89.0	2.35	5 (Brazil, Canada, Nordics)
C2	high-emissions mixed-grid, hyperscale-capacity	14.5	31.7	4.27	1 (China)
C3	mixed-grid, moderate-carbon, small-capacity	19.5	66.0	1.76	10 (France, Germany, UK, ...)
C4	high-emissions fossil-heavy, hyperscale-capacity	10.4	37.8	7.90	1 (United States)
C5	renewable-rich, low-carbon, small-capacity	81.1	100.0	2.08	1 (Iceland)

Table 3. K-Means centroids on the original feature scale, ordered by cluster index. Renewable share and low-carbon electricity are percentages; data-center capacity is the log1p-transformed centroid.

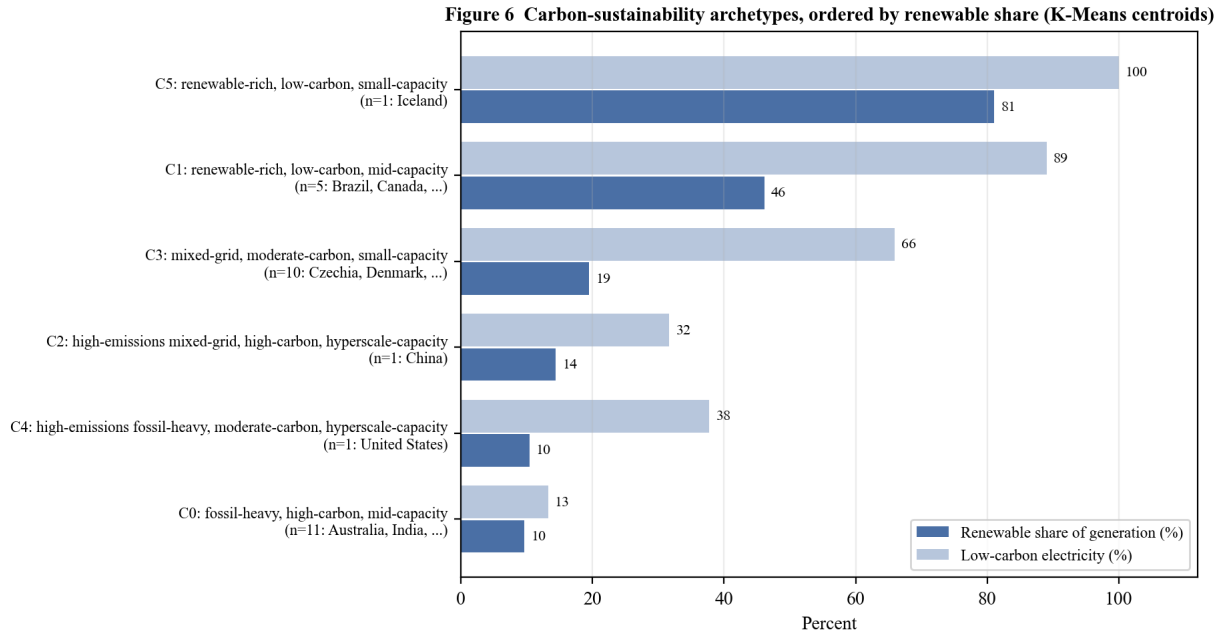


Figure 6. The six archetypes ordered by renewable share of generation, with each centroid's low-carbon-electricity percentage shown alongside. Cluster labels are derived from a rank-based tercile of the standardized centroid profile.

7. The Carbon-Opportunity Signal: From Predictions to a Recommendation

The three trained models are combined at inference time by `api/carbon_signal.py` into a single function, `green_window_probability(region, hour, day_of_week, month)`, which builds the same cyclical-plus-region-indicator feature row the classifier was trained on, queries the calibrated probability, and thresholds it into a plain-language recommendation:

- **P at or above 0.65:** RUN WORKLOAD, a high-confidence green window
- **0.40 up to 0.65:** WAIT, ambiguous, hold if the workload is deferrable
- **P below 0.40:** ADJUST, likely red window, shift time or region if possible

This mirrors the exact example the proposal specified as the target output format (Green Window Probability = 0.87; Recommendation = RUN WORKLOAD). When the caller does not supply a renewable percentage, the layer fills it from a region-by-hour climatological average (`renewable_climatology.csv`). Three additional functions add context: `facility_recommendation()` joins a specific ICARUS-DC facility to its grid region and returns the same signal annotated with operator, state, and power draw; `country_archetype()` looks up a country's K-Means cluster label and renewable share; and `predict_capacity_growth()` exposes the regressor directly as a coarse demand-side prioritization signal, with its documented limitations.

8. Serving the Signal: FastAPI and the OpenAPI (Swagger) Surface

The signal is exposed over HTTP by a FastAPI service (`api/main.py`) run locally with Uvicorn on port 8010. FastAPI generates an OpenAPI 3.1 schema from the route signatures and Pydantic models, and serves two interactive documentation surfaces from it with no extra code: Swagger UI at `/docs` and ReDoc at `/redoc`. The Swagger page (Figure 7) is what was used during development and the final presentation to exercise the endpoints by hand: expanding an operation, clicking “Try it out,” editing the JSON request body, and reading the live response and status code directly in the browser. Every request model carries field-level validation, so malformed scheduling requests fail fast with an HTTP 422 before they can reach a model.

Route	Method	Request	Description
/	GET	none	Health check and endpoint index
/regions	GET	none	The 12 valid EIA-930 grid region codes
/facilities	GET	none	ICARUS-DC facility lookup table (name, operator, state, region, MW)
/predict/window	POST	region, hour (1-24), day_of_week (0-6), month (1-12), renewable_pct (optional, 0-1)	Green Window Probability + RUN / WAIT / ADJUST recommendation
/predict/facility	POST	facility, hour, day_of_week, month	Same signal, resolved from an ICARUS-DC facility name to its grid region
/archetypes/{country}	GET	country (path param)	K-Means archetype label + renewable share + DC capacity for a country
/predict/growth	POST	year (2010-2030), overall_score (0-100), scale_score (0-100)	Regressor: predicted national DC capacity (MW)

The Carbon-Opportunity Signal API is titled “When and Where You Compute Is What You Emit” in its OpenAPI metadata. The full generated schema is reproduced in Appendix B.

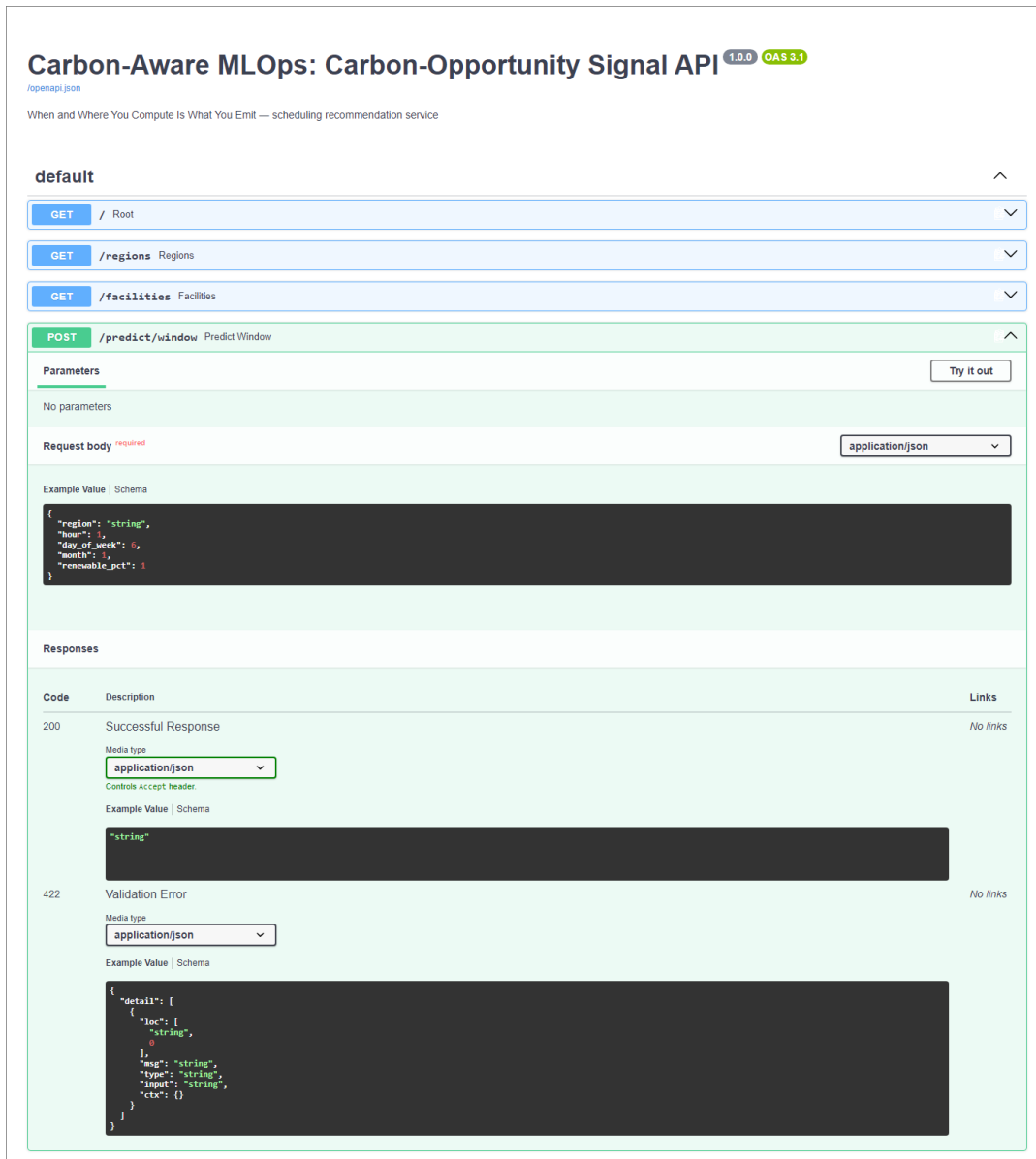


Figure 7. The live Swagger UI at /docs, with the POST /predict/window operation expanded to show the request-body schema (region, hour, day_of_week, month, optional renewable_pct) and the 200 / 422 responses. This is the interface used to hit the API by hand during development and the final demo.

9. Live API Demonstration

Figure 8 is the verbatim output of the scripted demo client (deployment/demo_api_calls.py) run against the local FastAPI instance. The client also issues the two GET calls (/ and /regions), which return the service index and the 12 region codes; the four prediction calls below are the substantive ones:

- **CAL, hour 14, Wednesday, July** - midday summer in California, when solar generation peaks. The layer fills renewable share from climatology at 0.68, the classifier returns a Green Window Probability of 0.9999, and the recommendation is RUN WORKLOAD. The region's fitted green threshold is 0.422

kg CO₂/kWh.

- **MIDW, hour 3, Saturday, January** - the pre-dawn Midwest in winter, a coal-heavy grid at low renewable output (0.19). Probability drops to 0.1804 and the recommendation is ADJUST: shift the workload in time or region if it is deferrable. The Midwest's green threshold, 1.054 kg CO₂/kWh, is more than double California's, which is exactly why the classifier uses region-relative labels.
- **Facility CVM, hour 14, Wednesday, July** - a small (0.12 MW) facility operated by CVM in Connecticut, resolved to the New England (NE) region. Probability 0.4103 lands in the WAIT band, and the response is annotated with the operator, state, and power draw pulled from the ICARUS-DC lookup.
- **Growth, year 2026, overall score 90, scale score 60** - the regressor returns a predicted national data-center capacity of 42.1 MW. Consistent with Section 6.2, this is surfaced only as a directional prioritization input, not a precise forecast.

```
$ python deployment/demo_api_calls.py http://127.0.0.1:8010
(health check and GET /regions omitted; shown below: the four prediction calls)

=====
POST http://127.0.0.1:8010/predict/window
Request body: {"region": "CAL", "hour": 14, "day_of_week": 2, "month": 7}
Status: 200
Response: {
  "region": "CAL",
  "hour": 14,
  "day_of_week": 2,
  "month": 7,
  "renewable_pct_used": 0.6824,
  "green_window_probability": 0.9999,
  "recommendation": "RUN WORKLOAD",
  "region_carbon_threshold_kg_co2_per_kwh": 0.422
}

=====
POST http://127.0.0.1:8010/predict/window
Request body: {"region": "MIDW", "hour": 3, "day_of_week": 5, "month": 1}
Status: 200
Response: {
  "region": "MIDW",
  "hour": 3,
  "day_of_week": 5,
  "month": 1,
  "renewable_pct_used": 0.1918,
  "green_window_probability": 0.1804,
  "recommendation": "ADJUST",
  "region_carbon_threshold_kg_co2_per_kwh": 1.054
}

=====
POST http://127.0.0.1:8010/predict/facility
Request body: {"facility": "CVM", "hour": 14, "day_of_week": 2, "month": 7}
Status: 200
Response: {
  "region": "NE",
  "hour": 14,
  "day_of_week": 2,
  "month": 7,
  "renewable_pct_used": 0.1419,
  "green_window_probability": 0.4103,
  "recommendation": "WAIT",
  "region_carbon_threshold_kg_co2_per_kwh": 0.504,
  "facility": "CVM",
  "operator": "CVM",
  "state": "Connecticut",
  "power_mw": 0.12
}

=====
POST http://127.0.0.1:8010/predict/growth
Request body: {"year": 2026, "overall_score": 90, "scale_score": 60}
Status: 200
Response: {
  "predicted_dc_power_capacity_mw": 42.1
}
```

Figure 8. Live output of `deployment/demo_api_calls.py` against the local Carbon-Opportunity Signal service. Each block shows the HTTP method, URL, request body, status code, and the full JSON response.

10. Deployment: Azure ML Managed Online Endpoint

The inference code in `carbon_signal.py` is deployment-target-agnostic by design: the identical logic backs both the local FastAPI development server and a production Azure ML Managed Online Endpoint, with only the hosting wrapper changing between the two. The Azure deployment is defined declaratively with two YAML files, applied with the Azure CLI:

```
$ az configure --defaults group=rg-carbon-mlops workspace=carbon-mlops-ws
$ az ml online-endpoint create -f endpoint.yml
$ az ml online-deployment create -f deployment.yml --all-traffic
```

Setting	Value
Workspace / resource group / region	carbon-mlops-ws / rg-carbon-mlops / East US
Endpoint name / auth mode	carbon-signal-endpoint / key
Deployment	blue (Standard_DS2_v2, 1 instance, 100% traffic)
Scoring URI	https://carbon-signal-endpoint.eastus.inference.ml.azure.com/score
Environment	conda.yaml on the AzureML openmpi/Ubuntu 20.04 base image
Entry script	deployment/score.py, standard <code>init()</code> / <code>run()</code> contract
Model asset	deployment/model_package/ (joblib artifacts + reference CSVs)

`score.py` implements Azure ML's `init()` / `run()` contract: `init()` loads the classifier, its feature list, and the per-region thresholds from `AZUREML_MODEL_DIR`, and `run()` accepts the same request body as `/predict/window` and returns the same signal. One concrete cross-version-compatibility bug surfaced during deployment and is worth recording: the per-region thresholds were originally pickled as a pandas `Series`, whose internal block manager can be pyarrow-backed and is not portable across pandas versions between the training machine and the Azure inference image. The fix was to persist a plain Python `dict` instead, an example of the kind of environment-parity issue that only appears once code crosses the boundary from notebook to managed runtime.

Two client-side consumption artifacts are checked in alongside the deployment config: a Postman collection (`deployment/carbon-signal.postman_collection.json`) with a saved request per endpoint and a `base_url` variable that points at either the local server or the Azure scoring URI, and the scripted demo client from Section 9. Both exercise the identical request/response contract regardless of where the model is hosted.

11. MLOps Lifecycle: Monitoring, Drift Detection, and Retraining

The proposal committed to a pipeline that supports “automated data ingestion, model training, deployment, monitoring, and retraining,” and the architecture in Figure 1 is built to make each of those a scheduled re-run of an existing script rather than a rewrite. The monitoring design has three parts:

- **Input drift.** The EIA-930 feed is refreshed continuously, and grid fuel mixes shift seasonally and structurally (new solar and wind capacity coming online, coal plants retiring). The ingest step recomputes each region's carbon-intensity distribution on every run, so a drift check compares the new per-region median and renewable-share climatology against the deployed model's fitted values; a large movement is the trigger to retrain rather than to silently keep serving.

- **Prediction drift.** The share of requests landing in each recommendation band (RUN / WAIT / ADJUST) is a cheap, model-agnostic health signal. A sudden collapse to almost-all-RUN or almost-all-ADJUST indicates either genuine grid change or a broken feature pipeline, and both warrant investigation before the recommendation is trusted.
- **Scheduled retraining.** Because `train_classifier.py` refits the region-relative thresholds on the training window each run, a periodic retrain keeps the green/red definition current with the evolving grid. Retraining regenerates the metrics JSON and the vetting figures, so a regression in held-out accuracy is visible before the new model is promoted.

Redeployment follows Azure ML's blue/green pattern: a retrained model is registered as a new deployment under the same endpoint, given a small traffic share for validation, and promoted to 100% only after its live metrics match or beat the incumbent. Because the endpoint URI and the request/response contract never change, downstream consumers, the Postman collection, the demo client, and any future workload orchestrator, are unaffected by a model swap.

12. Discussion: Limitations and Why Honest Reporting Matters

Three limitations are worth stating plainly. First, the regression component (Section 6.2) does not generalize to unseen countries at the target level, and the honest number should be the one anyone extending this work relies on, not the leaky 0.51. Second, two of the six originally-proposed datasets never became reality and were substituted with documented, justified alternatives (Section 4); the substitutions strengthened rather than weakened the pipeline, but the underlying lesson, that external data sources are not a static resource to build against, generalizes well beyond this project. Third, the clustering silhouette of 0.22, while sufficient to support six interpretable, business-usable archetypes, indicates real overlap between clusters; the three populous clusters in particular sit closer together in feature space than the three singleton clusters do, and a practitioner using the archetype label as a hard locational signal should treat it as directional rather than definitive.

A further limitation is that the classifier's renewable-share feature comes from historical climatology, not a live weather-driven generation forecast, so the model captures the *typical* shape of a region's day rather than the actual conditions of a specific future hour. Closing that gap is the highest-value item in future work.

What ties these limitations together is a single MLOps principle: a model's reported metric is only as trustworthy as the evaluation methodology that produced it. The gap between the naive-split and grouped-CV regression R^2 (0.51 versus 0.16) is, in a very literal sense, the most important number in this paper, not because the regressor performed well, but because it demonstrates that the same dataset and the same algorithm can be made to look nearly production-ready or clearly insufficient depending on how the split is constructed, and that a carbon-aware scheduling system deployed on the optimistic number would be shipping a false sense of confidence into production.

13. Evaluation, Feedback, and Outcomes

The project was graded at three checkpoints across the term: the Preliminary Proposal, the Algorithm Selection deliverable, and the Final Demonstration. All three received full marks (100/100). The instructor feedback is reproduced below.

“This is a good proposal that presents a unique and timely machine learning application with strong real-world relevance in sustainable AI and cloud computing. The proposal clearly defines the problem, identifies appropriate datasets, and explains how the prediction results can support carbon-aware workload scheduling in a practical way. The project demonstrates a strong understanding of the MLOps lifecycle by incorporating automated data ingestion, deployment, monitoring, drift detection, and retraining, while also defining measurable objectives and a realistic implementation timeline. The project is well scoped for the course and provides an excellent foundation for the algorithm selection and implementation phases.”

Instructor feedback, Preliminary Proposal (100/100)

“Nice job. This is a well-written and comprehensive submission with a clear justification for selecting XGBoost for the regression and classification tasks and K-Means for clustering. The rationale is well connected to the project's goals, and the discussion of algorithm comparisons, trade-offs, expected outputs, explainability, deployment, and operational considerations demonstrates a strong understanding of both machine learning and MLOps concepts. The report clearly explains how each model contributes to the overall solution and supports the project's decision-making process. Looking forward to the final project deliverables.”

Instructor feedback, Algorithm Selection (100/100)

“You did a great job building an end-to-end MLOps solution for carbon-aware workload scheduling. The project combined regression, classification, and clustering to turn carbon and infrastructure data into practical recommendations such as when and where workloads should run. The classification model met its targets with 86.1% accuracy and an F1 of 0.87, and you were also clear about the weaker regression results rather than overstating the model performance. The API demonstration showed how the models could be consumed as an actual service, which helped connect the modeling work to a practical use case. The discussion of data integration, model limitations, and how the different outputs were combined into actionable recommendations also strengthened the project. Overall, this was a solid application of the MLOps lifecycle from data and experimentation through deployment and consumption.”

Instructor feedback, Final Demonstration (100/100)

The instructor's specific callout of the honest regression reporting, in both the algorithm-selection and final feedback, rather than penalizing the missed target, validates the evaluation-methodology argument made in Section 12: in an MLOps context, transparently reporting a model's limits is treated as a strength of the system, not a weakness of the result.

14. Conclusion and Future Work

This project set out to answer a narrow but consequential question: can carbon-aware scheduling be operationalized as a deployable decision-support signal rather than left as exploratory analysis? It delivered a complete answer across all three stages of the MLOps lifecycle: data engineering resilient to source deprecation, three models trained and evaluated against explicit, falsifiable targets, and a fused signal served identically from a local FastAPI process with an interactive Swagger surface and an Azure ML Managed Online Endpoint. The classifier met its target outright; the regressor did not, and that result is reported as carefully as the successes, because in a system meant to inform real scheduling decisions, an honestly reported weak model is worth more than a leaky strong one.

Future work falls into three buckets: extending the classifier's cyclical feature set with true weather-driven renewable-generation forecasts rather than historical climatology; enriching the regression feature set with structural covariates (grid infrastructure age, GDP, hyperscaler capital-expenditure disclosures) that might close the gap the honest grouped-CV evaluation exposed; and closing the loop from recommendation to action by wiring the Carbon-Opportunity Signal directly into a workload orchestrator (for example, a Kubernetes scheduler plugin or an Airflow sensor) so that the RUN / WAIT / ADJUST recommendation is consumed automatically rather than read by a human.

This capstone was submitted as the final deliverable of the M.S. in Artificial Intelligence program at Johns Hopkins University, completed August 2026.

References

- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
- Electricity Maps. (2025). *Electricity Maps datasets: Hourly carbon intensity and electricity data*. <https://app.electricitymaps.com/datasets>
- HopkinsICARUS. (2025). *ICARUS-DC dataset* [Data set]. GitHub. <https://github.com/HopkinsICARUS/ICARUS-DC-Dataset>
- International Energy Agency. (2025). *Energy and AI*. IEA. <https://www.iea.org/reports/energy-and-ai>
- Kaggle. (2023). *Global data on sustainable energy (2000 to 2020)* [Data set]. <https://www.kaggle.com/datasets/anshtanwar/global-data-on-sustainable-energy>
- Kaggle. (2025). *AI readiness and growth, 150 countries* [Data set]. <https://www.kaggle.com/datasets/patelris/ai-readiness-and-growth-150-countries>
- Lawrence Berkeley National Laboratory. (2024). *2024 United States data center energy usage report*. U.S. Department of Energy.
- Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129-137.
- U.S. Energy Information Administration. (2026). *Hourly Electric Grid Monitor (EIA-930)*. <https://www.eia.gov/electricity/gridmonitor>

Appendix A: Reproducing the Pipeline

```
pip install -r requirements.txt

# 1. Ingest (raw downloads are gitignored; see docs/DATA_SOURCES.md)
python src/ingest_eia930.py
python src/ingest_ai_readiness.py
python src/ingest_clustering.py
python src/ingest_icarus.py

# 2. Train
python src/train_regression.py
python src/train_classifier.py
python src/train_clustering.py

# 3. Serve locally (Swagger UI at http://127.0.0.1:8010/docs)
cd api && uvicorn main:app --reload --port 8010

# 4. Demo the REST API
python deployment/demo_api_calls.py http://127.0.0.1:8010
```

Appendix B: Carbon-Opportunity Signal API Reference

Generated from the FastAPI OpenAPI 3.1 schema (/openapi.json).

Route	Method	Description
/predict/window	POST	Green Window Probability plus recommendation for a region / hour / day-of-week / month
/predict/facility	POST	Same signal, resolved from an ICARUS-DC facility name to its grid region, annotated with operator, state, and power draw
/predict/growth	POST	Regressor: predicted national data-center capacity (MW) from AI-adoption scores
/archetypes/{country}	GET	K-Means archetype label plus renewable share and DC capacity for a country
/regions	GET	The 12 valid EIA-930 grid region codes
/facilities	GET	ICARUS-DC facility lookup table
/	GET	Health check and endpoint index

Request validation. **WindowRequest:** region (string, one of the 12 codes), hour (integer 1-24), day_of_week (integer 0-6, 0 = Monday), month (integer 1-12), renewable_pct (optional float 0-1, overrides the climatological estimate). **GrowthRequest:** year (integer 2010-2030), overall_score and scale_score (floats 0-100). A value outside these bounds returns HTTP 422 with a field-level error before any model is called.

Appendix C: Repository Structure

```
carbon-aware-mlops/  
src/          ingest_*.py (4)    data ingestion + feature engineering  
              train_*.py (3)   XGBoost regression, XGBoost classifier, K-Means  
              make_figures.py  confusion matrix, importances, elbow, leakage plot  
data/processed/  model-ready tables (raw downloads gitignored)  
models/         3 .joblib artifacts + *_metrics.json  
api/           carbon_signal.py Carbon-Opportunity Signal (fuses the 3 models)  
              main.py         FastAPI service + OpenAPI / Swagger UI  
deployment/    score.py       Azure ML init() / run() scoring script  
              endpoint.yml, deployment.yml, conda.yaml  
              demo_api_calls.py, carbon-signal.postman_collection.json  
model_package/ registered model asset (artifacts + CSVs)  
docs/          DATA_SOURCES.md full provenance + documented substitutions  
              figures/        vetting figures used in the deck and this paper  
slides/        AIMLOps_Final_Presentation.pptx  
README.md, requirements.txt
```

The repository is version-controlled at

github.com/andrewsaifnoorian/carbon-aware-mlops. Commit history tracks the build in order: data ingestion, model training, the Carbon-Opportunity Signal and FastAPI service, the Azure ML deployment package, the cross-version pandas pickling fix, the vetting figures, and the final presentation deck.